

Documentation des Endpoints API

Endpoints d'Authentification Existants

1. Connexion (Login)

Endpoint: GET /api/login

Description: Connecte un utilisateur avec email et mot de passe.

Paramètres requis (GET): - email - Email de l'utilisateur - password - Mot de passe de l'utilisateur (sera hasché automatiquement)

URL d'exemple:

```
http://ptrniger.com/api/login?email=user@example.com&password=monmotdepasse
```

Réponse JSON (succès):

```
json
{
  "user_id": "123",
  "first_name": "John",
  "last_name": "Doe",
  "email": "user@example.com",
  "role": "user",
  "validity": 1,
  "device_verification": "no-need-verification",
  "token": "eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9..."
}
```

Réponse JSON (échec):

```
json
{
  "validity": 0,
  "device_verification": "invalid-login-credentials"
}
```

2. Inscription (Signup)

Endpoint: POST /api/signup

Description: Inscrit un nouvel utilisateur.

Paramètres requis (POST): - first_name - Prénom - last_name - Nom - email - Email - password - Mot de passe

Exemple avec JavaScript (FormData):

```
javascript
const formData = new FormData();
formData.append('first_name', 'John');
formData.append('last_name', 'Doe');
formData.append('email', 'user@example.com');
formData.append('password', 'monmotdepasse');

fetch('/api/signup', {
  method: 'POST',
  body: formData
})
.then(response => response.json())
.then(data => console.log(data));
```

Exemple avec URL-encoded (pour Postman):

```
javascript
fetch('/api/signup', {
  method: 'POST',
  headers: {
    'Content-Type': 'application/x-www-form-urlencoded'
  },
  body: 'first_name=John&last_name=Doe&email=user@example.com&password=monmotdepasse'
})
.then(response => response.json())
.then(data => console.log(data));
```

Support JSON et Form-Data : - **JSON** : Content-Type: application/json avec body JSON (maintenant supporté) - **Form-Data** : Onglet "Body" → "form-data" dans Postman - **URL-Encoded** : Onglet "Body" → "x-www-form-urlencoded" dans Postman

Exemple JSON pour Postman :

```
json
{
  "first_name": "John",
  "last_name": "Doe",
  "email": "user@example.com",
```

```
"password": "monmotdepasse"
}
```

Réponse JSON (succès):

```
json
{
  "message": "Registration successful",
  "email_verification": "enable",
  "status": 200,
  "validity": true
}
```

3. Vérification d'Email

Endpoint: POST /api/verify_email_address

Description: Vérifie l'adresse email avec le code reçu par email.

Paramètres requis (POST): - email - Email de l'utilisateur - verification_code - Code de vérification reçu par email

Réponse JSON (succès):

```
json
{
  "message": "Email verification successfully",
  "status": 200,
  "validity": true
}
```

4. Renvoyer le Code de Vérification

Endpoint: POST /api/resent_verification_code

Description: Renvoie le code de vérification par email.

Paramètres requis (POST): - email - Email de l'utilisateur

Réponse JSON (succès):

```
json
{
  "message": "Please check your inbox to verify your email address",
  "status": 200,
}
```

```
"validity": true
}
```

5. Mot de Passe Oublié

Endpoint: POST /api/forgot_password

Description: Envoie un lien de réinitialisation de mot de passe.

Paramètres requis (POST): - email - Email de l'utilisateur

Réponse JSON (succès):

```
json
{
  "message": "Successfully sent the verification link to your inbox",
  "status": 200,
  "validity": true
}
```

6. Confirmation de Connexion Multi-Appareils

Endpoint: POST /api/new_login_confirmation/{action}

Description: Gère la vérification lors de connexion depuis plusieurs appareils.

Actions possibles: - submit - Soumet le code de vérification - resend - Renvoie le code de vérification

Paramètres requis (POST): - auth_token - Token d'authentification - new_device_verification_code - Code de vérification (pour l'action submit)

Nouveaux Endpoints Créés

1. Récupérer toutes les catégories

Endpoint: GET /api/all_categories

Description: Retourne la liste de toutes les catégories avec leurs sous-catégories.

URL d'exemple:

```
http://ptrniger.com/api/all_categories
```

Réponse JSON:

```
json
[
  {
    "id": "1",
    "name": "Programmation",
    "parent": "0",
    "slug": "programmation",
    "font_awesome_class": "fas fa-code",
    "thumbnail": "programming.jpg",
    "sub_categories": [
      {
        "id": "2",
        "name": "PHP",
        "parent": "1",
        "slug": "php"
      },
      {
        "id": "3",
        "name": "JavaScript",
        "parent": "1",
        "slug": "javascript"
      }
    ]
  }
]
```

2. Récupérer tous les cours

Endpoint: GET /api/all_courses

Description: Retourne la liste de tous les cours actifs avec leurs détails complets.

URL d'exemple:

```
http://ptrniger.com/api/all_courses
```

Réponse JSON:

```
json
[
  {
    "id": "1",
    "title": "Formation PHP Avancée",
    "short_description": "Apprenez PHP de manière professionnelle",
```

```

    "description": "Description complète du cours...",
    "requirements": ["Connaissance de base en programmation"],
    "outcomes": ["Maîtrise de PHP", "Développement d'applications web"],
    "thumbnail": "http://ptrniger.com/uploads/thumbnails/course_thumbnails/course_1.jpg",
    "price": "99.99€",
    "rating": 4,
    "number_of_ratings": 25,
    "instructor_name": "John Doe",
    "instructor_image": "http://ptrniger.com/uploads/user_image/instructor_1.jpg",
    "total_enrollment": 150,
    "level": "advanced",
    "language": "french",
    "category_id": "1",
    "sub_category_id": "2",
    "status": "active",
    "shareable_link": "http://ptrniger.com/home/course/formation-php-avancee/1",
    "enable_drip_content": false
  }
]

```

Utilisation dans différents langages

JavaScript/Frontend

```

javascript
// Récupérer toutes les catégories
fetch('/api/all_categories')
  .then(response => response.json())
  .then(categories => {
    console.log('Catégories:', categories);
    // Traiter les catégories
  })
  .catch(error => console.error('Erreur:', error));

// Récupérer tous les cours
fetch('/api/all_courses')
  .then(response => response.json())
  .then(courses => {
    console.log('Cours:', courses);
    // Traiter les cours
  })
  .catch(error => console.error('Erreur:', error));

```

PHP

```

php
// Récupérer toutes les catégories

```

```
$categories = json_decode(file_get_contents('http://ptrniger.com/api/all_categories'), true);  
  
// Récupérer tous les cours  
$courses = json_decode(file_get_contents('http://ptrniger.com/api/all_courses'), true);
```

Python

```
import requests  
  
# Récupérer toutes les catégories  
response = requests.get('http://ptrniger.com/api/all_categories')  
categories = response.json()  
  
# Récupérer tous les cours  
response = requests.get('http://ptrniger.com/api/all_courses')  
courses = response.json()
```

cURL

```
bash  
# Récupérer toutes les catégories  
curl -X GET "http://ptrniger.com/api/all_categories"  
  
# Récupérer tous les cours  
curl -X GET "http://ptrniger.com/api/all_courses"
```

Endpoints d'Utilisateur Authentifié

Ces endpoints nécessitent un token d'authentification valide (auth_token).

1. Profil Utilisateur

Endpoint: GET /api/userdata?auth_token={token}

Description: Récupère les informations du profil utilisateur.

2. Mise à Jour du Profil

Endpoint: POST /api/update_userdata

Description: Met à jour les informations du profil utilisateur.

Paramètres requis (POST): - `auth_token` - Token d'authentification - `first_name` - Prénom - `last_name` - Nom - `phone` - Téléphone - `address` - Adresse - `biography` - Biographie

3. Changer le Mot de Passe

Endpoint: POST `/api/update_password`

Description: Change le mot de passe de l'utilisateur.

Paramètres requis (POST): - `auth_token` - Token d'authentification - `current_password` - Mot de passe actuel - `new_password` - Nouveau mot de passe

4. Mes Cours

Endpoint: GET `/api/my_courses?auth_token={token}`

Description: Récupère les cours auxquels l'utilisateur est inscrit.

5. Ma Liste de Souhaits

Endpoint: GET `/api/my_wishlist?auth_token={token}`

Description: Récupère la liste de souhaits de l'utilisateur.

6. Ajouter/Supprimer de la Liste de Souhaits

Endpoint: GET `/api/toggle_wishlist_items?auth_token={token}&course_id={id}`

Description: Ajoute ou supprime un cours de la liste de souhaits.

Endpoints API Existants

Le système dispose également d'autres endpoints utiles :

- GET `/api/top_courses` - Récupère les cours mis en avant
- GET `/api/categories/{category_id}` - Récupère une catégorie spécifique
- GET `/api/sub_categories/{parent_category_id}` - Récupère les sous-catégories
- GET `/api/category_wise_course?category_id={id}` - Récupère les cours d'une catégorie
- GET `/api/filter_course` - Filtre les cours selon des critères
- GET `/api/courses_by_search_string?search_string={terme}` - Recherche des cours

Codes de Réponse HTTP

- **200 OK** - Succès
- **404 Not Found** - Endpoint non trouvé
- **405 Method Not Allowed** - Erreur de nommage d'endpoint (voir section Dépannage)
- **500 Internal Server Error** - Erreur serveur

Dépannage des Erreurs Communes

Erreur 405 "Unknown method"

Problème : {"status":false,"error":"Unknown method"}

Cause : URL incorrecte avec suffixe de méthode HTTP

Incorrect :

```
GET /api/login_get
POST /api/signup_post
```

Correct :

```
GET /api/login
POST /api/signup
```

Explication : Le REST Controller de CodeIgniter combine automatiquement : - URL demandée (/api/login) + Méthode HTTP (GET) = Méthode exécutée (login_get())

Support JSON Amélioré

Bonne nouvelle : Le support JSON a été ajouté aux endpoints d'authentification !

Maintenant supporté : - Content-Type: application/json avec body JSON - Content-Type: application/x-www-form-urlencoded - multipart/form-data (form-data)

Endpoints avec support JSON : - POST /api/signup - POST /api/verify_email_address - POST /api/resend_verification_code - POST /api/forgot_password - POST /api/update_userdata - POST /api/update_password

Gestion des Tokens d'Authentification

Comment obtenir un token

1. **Connexion réussie** : Le token est fourni dans la réponse de `/api/login_get`
2. **Stockage** : Stocker le token de manière sécurisée côté client
3. **Utilisation** : Inclure le token dans les requêtes nécessitant une authentification

Exemple d'utilisation du token

```
javascript
// Après connexion réussie
const loginResponse = await fetch('/api/login?email=user@example.com&password=motdepasse');
const userData = await loginResponse.json();

if (userData.validity === 1) {
    const token = userData.token;

    // Utiliser le token pour des requêtes authentifiées
    const coursesResponse = await fetch(`/api/my_courses?auth_token=${token}`);
    const myCourses = await coursesResponse.json();
}
```

Gestion de l'expiration

- Les tokens ont une durée de vie limitée
- Surveillez les réponses d'erreur 401 (Non autorisé)
- Reconnectez l'utilisateur si le token expire

Notes Importantes

1. **Cours actifs uniquement** : L'endpoint `all_courses_get` ne retourne que les cours avec le statut "active".
2. **Données enrichies** : Les données des cours incluent : - Thumbnails avec URLs complètes - Prix formatés selon la devise du système - Ratings calculés - Informations sur l'instructeur - Nombre d'inscriptions - Liens partageables
3. **Performance** : Ces endpoints peuvent retourner beaucoup de données. Pour de gros volumes, considérez l'ajout de pagination.

4. **Sécurité** : - Les endpoints publics ne nécessitent pas d'authentification - Les endpoints d'utilisateur authentifié nécessitent un token valide - Les mots de passe sont automatiquement hachés (SHA1)

5. **Vérification d'email** : Le système peut être configuré pour nécessiter une vérification d'email lors de l'inscription.

Test des Endpoints

Utilisez le fichier `test_api_endpoints.php` pour tester rapidement les nouveaux endpoints depuis votre navigateur.

Structure des Fichiers Modifiés

- `application/models/Api_model.php` - Ajout de la méthode `all_courses_get()`
- `application/controllers/Api.php` - Ajout de l'endpoint `all_courses_get()`